
Lab 2

Reflection Report

Data Pipeline with dbt & DuckDB

Hamza Laraisse

Data Engineering Lab

Technology Stack: dbt Core, DuckDB, Power BI

Data Source: Google Play Store API

Schema Design: Kimball Star Schema

Course: Data Engineering

Contents

1	Dimensional Data Modeling	2
1.1	Business Process	2
1.2	Grain Declaration	2
1.3	Dimensions	2
1.4	Facts (Measures)	3
1.5	Bus Matrix	3
1.6	Star Schema Design	3
1.6.1	Fact Table: <code>fact_reviews</code>	4
1.6.2	Validation Against Analytical Needs	4
2	Final Architecture	4
2.1	Overview	4
2.2	Project Folder Structure	5
2.3	dbt Configuration	5
2.3.1	profiles.yml	5
2.3.2	dbt_project.yml	6
2.3.3	Schema Naming Behavior	6
2.4	Power BI Logical Model	7
3	Key Implementations	8
3.1	Incremental Loading	8
3.2	Slowly Changing Dimensions — SCD Type 2	9
3.3	Data Quality & Testing	11
4	Python-Only vs dbt-Based Pipeline Comparison	11
5	Reflections	12
5.1	What Was the Most Fragile Part of the Pipeline?	12
5.2	What Was the Biggest Architectural Insight Gained?	12
5.3	What Is One Design Decision You Would Change?	12
5.4	Power BI Dashboards	13

1. Dimensional Data Modeling

1.1. Business Process

App Reviews on the Google Play Store

The recurring, measurable event we want to analyze is a **user submitting a review** (a rating and optional text) for an application on the Google Play Store. This business process captures:

- **What** happened: a review was posted
- **Which** app was reviewed
- **Who** developed the app
- **When** the review was submitted
- **How** the user rated the app (numeric score, thumbs up)

1.2. Grain Declaration

“One row in the fact table represents one user review submitted for a specific app, developed by a known developer, on a specific date.”

Each row in `fact_reviews` corresponds to exactly one review event, uniquely identified by `review_id`, and linked to a specific app, developer, and date through foreign keys.

1.3. Dimensions

Dimension	Business Meaning	Key Attributes
dim_apps	Describes the app being reviewed	app_id, app_name, price, is_paid, installs, catalog_rating, ratings_count
dim_categories	The app's genre or category	category_key, category_name
dim_developers	The developer behind the app	developer_key, developer_name, developer_website, developer_email
dim_date	Time context of the review	date, year, month, quarter, day_of_week, is_weekend

1.4. Facts (Measures)

Measure	Type	Aggregations	Notes
rating	INTEGER	AVG, MIN, MAX	Core review score (1–5)
thumbs_up_count	INTEGER	SUM, AVG	Helpfulness signal
review_text	TEXT	COUNT (non-null)	Descriptive; used for text analysis
review_version	VARCHAR	GROUP BY	App version at review time

1.5. Bus Matrix

The Bus Matrix maps each business process to the dimensions it can be analyzed by, acting as the bridge between business requirements and the physical star schema.

Business Process	dim_apps	dim_categories	dim_developers	dim_date
App Reviews	+	+	+	+

Design note: `dim_categories` is reached *indirectly* through `dim_apps` (via `category_key` in `dim_apps`). This is a minor denormalization trade-off: since an app always belongs to a fixed category, the join path is short and efficient.

1.6. Star Schema Design

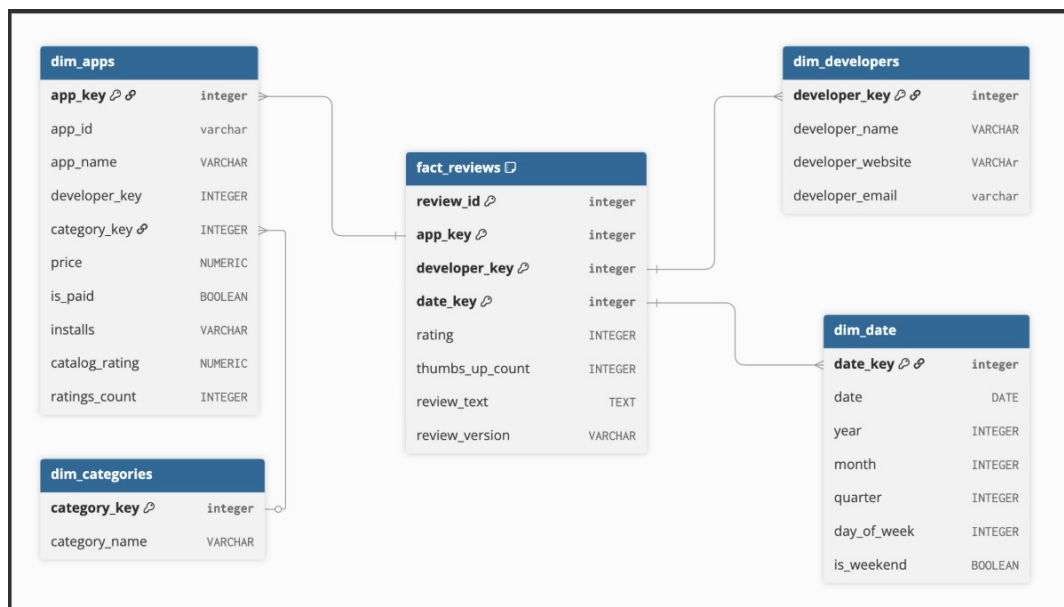


Figure 1: Star Schema — Google Play Store Reviews. `fact_reviews` sits at the center linked to four conformed dimension tables via surrogate keys.

1.6.1 Fact Table: fact_reviews

Column	Type	Description
review_id (PK)	INTEGER	Surrogate primary key
app_key (FK)	INTEGER	Foreign key to dim_apps
developer_key (FK)	INTEGER	Foreign key to dim_developers
date_key (FK)	INTEGER	Foreign key to dim_date
rating	INTEGER	User rating score (1–5)
thumbs_up_count	INTEGER	Number of helpful votes
review_text	TEXT	Full text of the review
review_version	VARCHAR	App version at time of review

1.6.2 Validation Against Analytical Needs

Analytical Question	How the Model Answers It
What is the average rating per app?	AVG(rating) GROUP BY dim_apps.app_name
Which category gets the most reviews?	COUNT(review_id) GROUP BY dim_categories.category_name
How do ratings evolve over time?	AVG(rating) GROUP BY dim_date.month, year
Which developer has the best-rated apps?	AVG(rating) GROUP BY dim_developers.developer_name
Do paid apps get better reviews?	AVG(rating) GROUP BY dim_apps.is_paid
Which app version has most thumbs up?	SUM(thumbs_up_count) GROUP BY review_version
Are weekend reviews rated differently?	AVG(rating) GROUP BY dim_date.is_weekend

2. Final Architecture

2.1. Overview

The pipeline re-engineers the Lab 1 Python-only approach into a fully modular, framework-driven architecture. It is composed of four distinct layers, each with a clearly defined responsibility:

- **Ingestion Layer** — Python script (`capture_lab2.py`) calls the Google Play Store API using `google-play-scraper`, collecting app metadata and paginated reviews, persisting them as immutable JSONL files in the `data/` folder.
- **Staging Layer** — Two dbt view models (`stg_playstore_apps`, `stg_playstore_reviews`) read the raw JSONL files using DuckDB's `read_json_auto()` function, standardize column names, cast data types, and filter null keys.
- **Marts Layer** — Four dimension tables (`dim_apps`, `dim_categories`, `dim_developers`, `dim_date`) and one fact table (`fact_reviews`) implement the Kimball star schema.

- **Serving Layer** — Power BI connects to DuckDB via an ODBC driver, consuming the materialized tables directly for reporting and visualization.

2.2. Project Folder Structure

The dbt project (named `save`) is organized as follows:

```
DBT_project/
  data/
    apps_metadata.jsonl      <- Raw app data (ingestion output)
    apps_reviews.jsonl      <- Raw reviews data (ingestion output)
  duckdb/
    analytics.duckdb        <- DuckDB analytical database
  models/
    staging/
      stg_playstore_apps.sql
      stg_playstore_reviews.sql
    marts/
      dimensions/
        dim_apps.sql
        dim_categories.sql
        dim_developers.sql
        dim_date.sql
      facts/
        fact_reviews.sql    <- Incremental model
  snapshots/
    snap_apps.sql          <- SCD Type 2 snapshot
```

2.3. dbt Configuration

Two configuration files govern how dbt connects to DuckDB and how models are materialized. Getting these right was a critical setup step — a mismatch between the profile name in `profiles.yml` and `dbt_project.yml` caused models to silently fall back to default settings, which is why dimensions were initially not appearing.

2.3.1 profiles.yml

The `profiles.yml` file tells dbt how to connect to the database. It must be named identically to the profile key in `dbt_project.yml` (both set to `save`):

```
save:
  outputs:
    dev:
      type: duckdb
      path: duckdb/analytics.duckdb
      threads: 4
    prod:
      type: duckdb
      path: duckdb/prod.duckdb
      threads: 4
  target: dev
```

The `path` field points to the DuckDB file relative to the project root. The `threads: 4` setting enables parallel model execution. The `target: dev` sets the active environment.

2.3.2 dbt_project.yml

The `dbt_project.yml` file defines materialization conventions per folder, enforcing a clean separation between lightweight staging transformations (views) and analytics-ready mart tables (materialized tables):

```
name: 'DBT_project'
version: '1.0.0'

profile: 'DBT_project'

model-paths: ["models"]
analysis-paths: ["analyses"]
test-paths: ["tests"]
seed-paths: ["seeds"]
macro-paths: ["macros"]
snapshot-paths: ["snapshots"]

clean-targets:
  - "target"
  - "dbt_packages"

models:
  save:
    staging:
      +materialized: view
      +schema: staging
  marts:
    +materialized: table
    dimensions:
      +materialized: table
      +schema: dimensions
    facts:
      +materialized: table
      +schema: facts
```

2.3.3 Schema Naming Behavior

An important behavior to understand is how dbt generates schema names. When a `+schema` value is set, dbt **appends** it to the default target schema rather than replacing it. Since DuckDB's default schema is `main`, setting `+schema: staging` produces `main_staging`, not `staging`. This explains the schema structure visible in Power BI:

dbt Folder	DuckDB Schema	Materialization
staging	main_staging	view
dimensions	main_dimensions	table
facts	main_facts	table (incremental)
snapshots	main_snapshots	snapshot

This behavior can be overridden by adding a `generate_schema_name` macro, but for this project the `main_` prefix convention was kept as it clearly identifies each layer in both DuckDB and Power BI.

2.4. Power BI Logical Model

Once the marts layer was built, Power BI was connected to `analytics.duckdb` via a DuckDB ODBC driver. The screenshot below shows the final logical model as detected and configured in Power BI, with all five tables and their relationships correctly mapped.

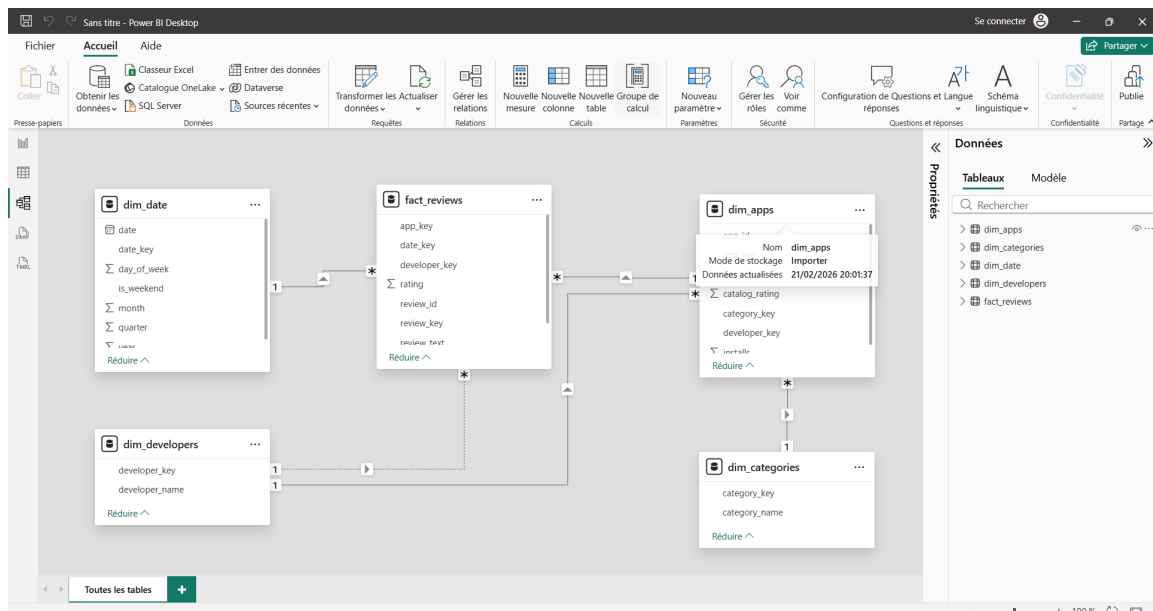


Figure 2: Power BI logical model showing `fact_reviews` at the center, linked to `dim_apps`, `dim_categories`, `dim_developers`, and `dim_date` via surrogate keys. The `dim_apps` to `dim_categories` relationship (1-to-many) reflects the app hierarchy implemented in the dimensional model.

3. Key Implementations

3.1. Incremental Loading

The `fact_reviews` model was initially materialized as a full `table`, meaning every `dbt run` would drop and rebuild the entire table from scratch. This approach is inefficient as data volumes grow, since it reprocesses rows that have already been loaded.

To address this, the model was refactored to use `dbt`'s `incremental` materialization with the following configuration:

```
{{ config(
    materialized='incremental',
    unique_key='review_id',
    on_schema_change='fail'
) }}
```

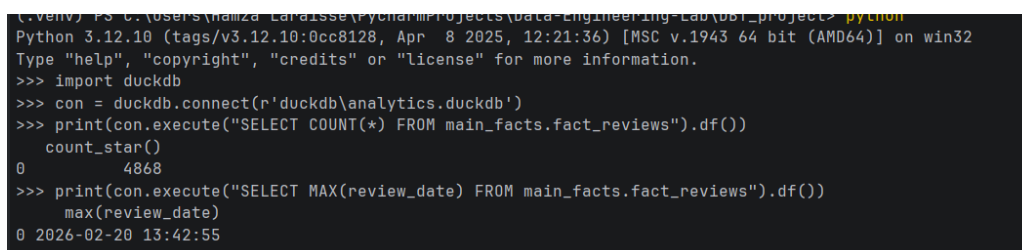
The incremental logic filters the source data using `dbt`'s `is_incremental()` macro. To avoid a DuckDB limitation where aggregates are not allowed directly in `WHERE` clauses, the maximum existing date is first computed in a dedicated CTE:

```
{% if is_incremental() %}
max_date as (
    select max(review_date) as last_review_date
    from {{ this }}
),
{% endif %}

reviews as (
    select * from {{ ref('stg_playstore_reviews') }}
    where review_id is not null
    {% if is_incremental() %}
        and review_date > (select last_review_date from max_date)
    {% endif %}
)
```

On the first run, the table is built in full. On subsequent runs, only reviews with a `review_date` strictly greater than the current maximum are processed and inserted.

Proof of incremental behavior: The screenshots below show the row count and maximum review date before and after an incremental run following a new data ingestion.



```
(.venv) PS C:\Users\hamza.Laraisse\PycharmProjects\data-engineering-lab\dbt_project> python
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license" for more information.
>>> import duckdb
>>> con = duckdb.connect(r'duckdb\analytics.duckdb')
>>> print(con.execute("SELECT COUNT(*) FROM main_facts.fact_reviews").df())
count_star()
0      4868
>>> print(con.execute("SELECT MAX(review_date) FROM main_facts.fact_reviews").df())
max(review_date)
0 2026-02-20 13:42:55
>>>
```

Figure 3: State before incremental run: 4,868 rows, max review date 2026-02-20 13:42:55.

```

20:37:20 Done. PASS=1 WARN=0 ERROR=0 SKIP=0 NO-OP=0 TOTAL=1
(.venv) PS C:\Users\Hamza Laraisse\PycharmProjects\Data-Engineering-Lab\DBT_project> python
Python 3.12.10 (tags/v3.12.10:0cc8128, Apr 8 2025, 12:21:36) [MSC v.1943 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>> import duckdb
>>> con = duckdb.connect(r'duckdb\analytics.duckdb')
>>> print(con.execute("SELECT COUNT(*) FROM main_facts.fact_reviews").df())
count_star()
0      4935
>>> print(con.execute("SELECT MAX(review_date) FROM main_facts.fact_reviews").df())
max(review_date)
0  2026-02-20 20:34:29

```

Figure 4: State after incremental run: 4,935 rows, max review date 2026-02-20 20:34:29. Only the new reviews were inserted — the existing 4,868 rows were untouched. The run completed in 0.43 seconds, confirming no full rebuild occurred.

3.2. Slowly Changing Dimensions — SCD Type 2

In the initial design, `dim_apps` represented only the current state of each app. Any change to an app's attributes (such as its category) would silently overwrite the previous value, making historical analysis impossible. SCD Type 2 solves this by preserving all historical versions of a record.

The implementation uses dbt's native snapshot functionality with the `check` strategy, which monitors a defined list of columns for changes:

```

{% snapshot snap_apps %}
{{
    config(
        target_schema='snapshots',
        unique_key='app_id',
        strategy='check',
        check_cols=['app_name', 'catalog_rating', 'installs',
                    'category_key', 'developer_key']
    )
}}
select app_id, app_name, catalog_rating, installs,
       category_key, developer_key
from {{ ref('dim_apps') }}
{% endsnapshot %}

```

On top of the snapshot, a clean `dim_apps_scd` model was created to expose the history with human-readable validity columns and a convenience `is_current` flag:

```

select
    md5(coalesce(app_id, '') || coalesce(cast(dbt_valid_from as
        varchar), ''))
        as app_scd_key,
    app_id, app_name, catalog_rating, installs, category_key,
    developer_key,
    dbt_valid_from as valid_from,
    dbt_valid_to as valid_to,
    case when dbt_valid_to is null then true else false end as
        is_current
from {{ ref('snap_apps') }}

```

The screenshot below shows the full DuckDB schema structure as visible in Power BI's Navigator after the complete pipeline run, including the `snap_apps` table in the `snapshots` schema:

The screenshot shows the Power BI Navigator interface. On the left, the 'Navigator' pane displays the DuckDB schema structure under 'ODBC (dsn=duckdb_playstore) [1]'. The schema is organized into several folders: 'analytics [5]', 'main [1]', 'main_dimensions [5]', 'main_facts [1]', 'main_staging [3]', and 'snapshots [1]'. The 'snapshots' folder is expanded, showing the 'snap_apps' table. On the right, the 'snap_apps' table is previewed, showing columns: 'app_id', 'app_name', 'catalog_rating', and 'installs'. The table contains multiple rows, including historical versions of the same app (e.g., 'ai.turbolearn', 'ai.fireflies.mobile') which are preserved due to the SCD Type 2 mechanism.

app_id	app_name	catalog_rating	installs
com.codespaceapps.meetingtranscriber	Smart Noter - AI Note Taker	65014	6675
ai.turbolearn	Turbo AI - Notetaker	258183	1988
ai.fireflies.mobile	Fireflies: AI notetaker	4402	312
com.hubx.noteai	Note AI: Smart Note Taker	8483	404
com.studocu.app	Studocu: AI Notes & Summaries	22550	2956
com.microsoft.office.onenote	Microsoft OneNote: Save Notes	1463494	971455
com.yygg.note.app	Notewise - AI Notes, PDF, Docs	25003	6041
ai.note.notepad.memo.journal.freemote	Freemote - Notes Taking & PDF	7264	651
io.labhouse.audioapp	Summary - AI Note Taker	33527	661
com.sota.ai.notex	NoteX: AI Notes & Meetings	1909	123
com.aisense.otter	Otter AI Transcribe Voice Notes	25953	5760
com.recordpro.audiorecord	HiNoter - AI Note Taker	4367	271
notion.id	Notion: Notes, Tasks, AI	337965	34372
ai.meeting.app	Meeting.ai: AI Visual Notes	12203	378
com.google.android.keep	Google Keep - Notes and lists	2283831	3648991
com.ma.wavesapp	Wave AI Note Taker & Recorder	2029	165
ai.notes.notebook	HyNote: AI Notebook LLM Notes	3395	174
com.feraset.ainote	AI Note Taker - Meeting Notes	1612	40
com.meetgeek.assistant	AI Transcribe. Speech to Text	919	228
com.google.android.apps.labs.language.tailwind	Google NotebookLM	143311	13141
com.codespaceapps.meetingtranscriber	Smart Noter - AI Note Taker	65029	6675
ai.turbolearn	Turbo AI - Notetaker	258227	1988
ai.fireflies.mobile	Fireflies: AI notetaker	4404	312

Figure 5: Power BI Navigator showing the complete DuckDB database structure. The database is organized into four schemas: `main_dimensions` (containing `dim_apps`, `dim_apps_scd`, `dim_categories`, `dim_date`, `dim_developers`), `main_facts` (containing `fact_reviews`), `main_staging` (staging views), and `snapshots` (containing `snap_apps`). The preview panel displays the `snap_apps` snapshot table, where duplicate `app_id` entries (e.g., `ai.turbolearn`, `ai.fireflies.mobile`) are visible — these are the historical versions created by the SCD Type 2 mechanism, confirming that multiple versions of the same app are correctly preserved.

Proof of SCD2 behavior: To test the implementation, the `genre` field of the app `ai.fireflies.mobile` was modified in the raw JSONL file. After re-running the staging, dimension, and snapshot models, the query below reveals two distinct versions of the same app:

```
>>> import duckdb
>>> con = duckdb.connect(r'duckdb\analytics.duckdb')
>>> print(con.execute("""
...     SELECT app_id, app_name, category_key, valid_from, valid_to, is_current,
...     FROM main_dimensions.dim_apps_scd
...     WHERE app_id = 'ai.fireflies.mobile'
...     ORDER BY valid_from
... """).df())
```

	app_id	app_name	category_key	valid_from	valid_to	is_current
0	ai.fireflies.mobile	Fireflies: AI notetaker	73015ecba0f4f12bb5a49cdd0796137a	2026-02-21 20:59:00.666251	2026-02-21 21:01:13.687482	False
1	ai.fireflies.mobile	Fireflies: AI notetaker	8625e1de7be14c39b1d14dc03d822497	2026-02-21 21:01:13.687482	NaT	True

Figure 6: SCD2 output for `ai.fireflies.mobile`. Row 0 (historical) shows the old `category_key` with a closed `valid_to` timestamp and `is_current` = False. Row 1 (current) shows the new `category_key` with `valid_to` = NaT and `is_current` = True. Full history is preserved — nothing was overwritten.

3.3. Data Quality & Testing

dbt's generic test framework was used throughout all three layers to enforce data quality automatically on every pipeline run.

Model	Test	Purpose
stg_apps	unique, not_null on <code>app_id</code>	Ensures no duplicate or missing app identifiers at ingestion
stg_reviews	unique, not_null on <code>review_id</code>	Guarantees grain integrity in the staging layer
dim_categories	unique, not_null on <code>category_key</code>	Validates surrogate key uniqueness
dim_apps	relationships to <code>dim_categories</code>	Referential integrity: every app must have a valid category
dim_apps	relationships to <code>dim_developers</code>	Referential integrity: every app must have a valid developer
fact_reviews	relationships to all 3 dimensions	Foreign key integrity across the entire star schema
fact_reviews	accepted_values on <code>rating</code> (1–5)	Domain validation: ratings must be within valid range

In total, **43 tests** were defined across all models, run automatically with `dbt test`.

4. Python-Only vs dbt-Based Pipeline Comparison

Aspect	Python-Only (Lab 1)	dbt-Based (Lab 2)
Transformation logic	Ad-hoc Python scripts	Declarative SQL models with <code>ref()</code> lineage
Data quality	Manual assertions in code	43 automated schema tests
Reproducibility	Depends on script execution order	Enforced by dbt DAG
Incremental loading	Manual state tracking	Native <code>is_incremental()</code> macro
Historical tracking	Not supported	SCD2 via dbt snapshots
Schema changes	Silent failures	<code>on_schema_change='fail'</code> guard
Documentation	In-code comments	<code>schema.yml</code> descriptions on every model
BI serving	Pandas DataFrames	DuckDB tables via ODBC

The most significant practical difference is **reproducibility**. In Lab 1, re-running a script in the wrong order could silently corrupt or duplicate data. In the dbt-based approach, the DAG enforces the correct execution order automatically, and the `-full-refresh` flag provides a clean rebuild path when needed.

5. Reflections

5.1. What Was the Most Fragile Part of the Pipeline?

The most fragile component was the **DuckDB file lock**. DuckDB operates as a file-based database without a server process, which means only one process can hold the file open at a time. Throughout the lab, running dbt while Power BI was connected to `analytics.duckdb` consistently caused errors such as:

```
I0 Error: Cannot open file "analytics.duckdb": The process cannot
access the file because it is being used by another process.
File is already open in Microsoft.Mashup.Container.NetFX45.exe
```

This required manually closing Power BI before every `dbt run`, which is not sustainable in a production environment. In a real deployment, this would be addressed by using a server-based database (such as PostgreSQL or MotherDuck) that supports concurrent connections.

5.2. What Was the Biggest Architectural Insight Gained?

The biggest insight was understanding the **value of separating concerns across pipeline layers**. In Lab 1, the Python scripts mixed ingestion, transformation, and serving logic, making debugging difficult and changes risky. In the dbt architecture, each layer has a single, well-defined responsibility:

- Staging models act as a *contract* between raw data and business logic.
- Dimension models are *conformed* and reusable across multiple fact tables.
- The fact table joins *only on surrogate keys*, never on raw natural keys.

This separation meant that when column names in the raw data were discovered to be different from what was expected (e.g., `genre` instead of `category`, `score` instead of `rating`), only the staging model needed to be updated. The dimension and fact models were completely unaffected, demonstrating the resilience of a properly layered architecture.

5.3. What Is One Design Decision You Would Change?

The **surrogate key strategy** would be revised. Throughout the project, surrogate keys were generated using `md5(coalesce(column, ''))`. While functional, this approach has two weaknesses:

1. **Collision risk:** MD5 is not collision-resistant enough for very large datasets. A SHA-256 or UUID-based approach would be safer.
2. **Null handling:** `coalesce(column, '')` maps null values to an empty string, meaning a null `app_id` and an empty string `app_id` would produce the same key. This should be handled more explicitly, for example by raising an error upstream if the natural key is null (which our `not_null` tests partially enforce).

The preferred production approach would be to use `dbt-utils.generate_surrogate_key()` with proper null handling. This was initially attempted but required installing the `dbt-utils` package, which was not available in the project environment.

5.4. Power BI Dashboards

The final step of the lab connected the DuckDB star schema to Power BI via ODBC, enabling the creation of two key analytical visualizations from the `fact_reviews` table:

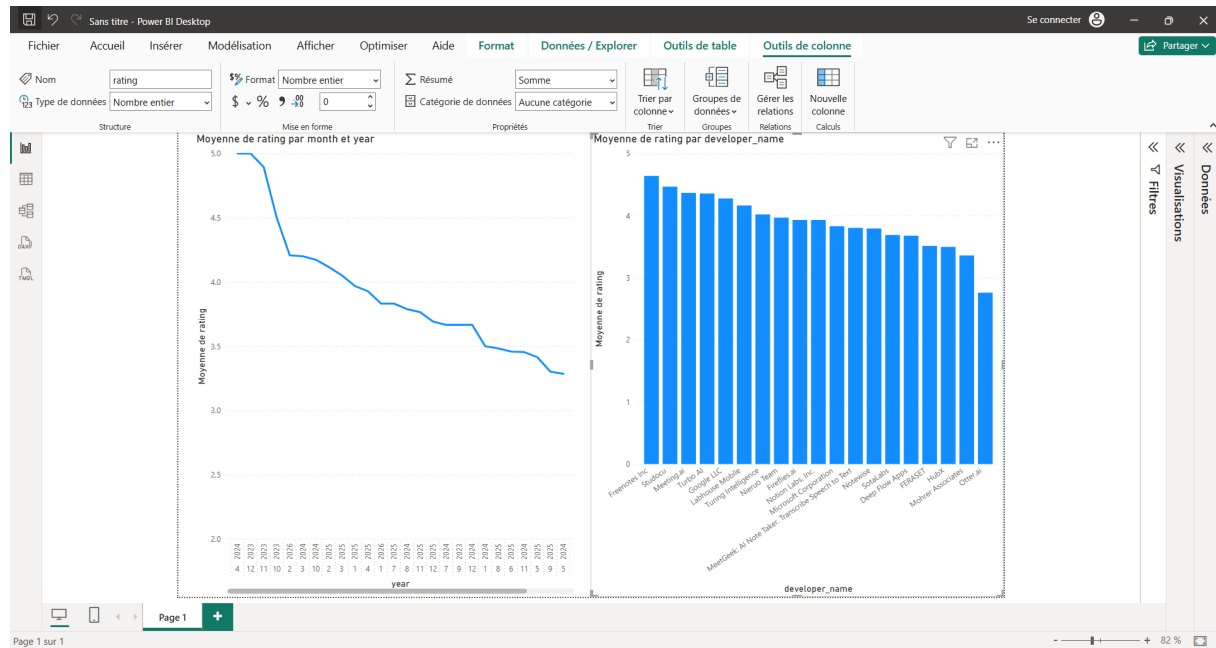


Figure 7: Power BI dashboards built on top of the star schema. *Left*: Monthly average rating trend over time, showing a gradual decline in average ratings from 5.0 in early 2024 to approximately 3.3 by early 2026 — suggesting increasing user expectations or app quality degradation over time. *Right*: Developer performance comparison by average rating, with Freenotes Inc. leading at approximately 4.5 and Otter.ai at the lower end near 2.8.